```
module M
end

orphan = Object.send(:remove_const, :M)

# The module object is orphan now but it still has a name.
orphan.name # => "M"

# You cannot reach it via the constant M because it does not even
exist.
orphan.reachable? # => false

# Let's define a module called "M" again.
module M
end

# The constant M exists now again, and it stores a module
# object called "M", but it is a new instance.
orphan.reachable? # => false
```

Defined in `active_support/core_ext/module/reachable.rb`.

3.6 Anonymous

A module may or may not have a name:

```
module M
end
M.name # => "M"

N = Module.new
N.name # => "N"

Module.new.name # => nil
```

You can check whether a module has a name with the predicate `anonymous?`:

```
module M
end
M.anonymous? # => false

Module.new.anonymous? # => true
```

Note that being unreachable does not imply being anonymous:

```
module M
end

m = Object.send(:remove_const, :M)

m.reachable? # => false
m.anonymous? # => false
```

though an anonymous module is unreachable by definition.

Defined in `active_support/core_ext/module/anonymous.rb`.

3.7 Method Delegation

The macro `delegate` offers an easy way to forward methods.

Let's imagine that users in some application have login information in the `User` model but name and other data in a separate `Profile` model:

```
class User < ActiveRecord::Base
```